

shap_lime_explainer.ipynb – Technical Report

Model Explainability Review (SHAP & LIME) — Loan Default Prediction (XAI) Project

1. Purpose of This File in the Project

shap_lime_explainer.ipynb is the explainability stage of the pipeline, sitting after model training (building_models.ipynb) and before/alongside the deployment layer (app.py). Its job is to turn the trained LightGBM classifier's raw predictions into human-interpretable explanations that a non-technical end user can trust. Concretely, it is responsible for:

- Loading the persisted LightGBM model and the held-out X_test / y_test splits produced earlier in the pipeline.
- Computing SHAP values on a representative sample of the test set to identify global feature importance.
- Visualizing both the magnitude (bar chart) and direction (beeswarm) of each feature's impact on default risk.
- Producing a per-customer waterfall breakdown for a single, concrete prediction.
- Converting raw SHAP numbers into a plain-English explanation sentence intended for direct reuse in app.py.
- Cross-checking SHAP's explanation against LIME, an independently-built local explanation method, as a sanity check.

Compared to Preprocessing.ipynb, this notebook is small and narrowly scoped (9 sections, 22 cells) and runs end-to-end without errors. Its main value is bridging the gap between a raw model probability ("78% default risk") and the specific reasons behind it, which is exactly what the deployed app needs to show end users.

2. Notebook Context & Inputs (Notebook Introduction)

The notebook opens by stating it explains the project's best model — LightGBM, reported elsewhere as ROC-AUC = 0.74 — using two complementary tools:

Tool	What It Tells Us
SHAP	How much each feature pushed an individual prediction up (toward default) or down (away from default).
LIME	A second, independently-built opinion on the same prediction, used to sanity-check SHAP.

The two inputs it consumes, both produced by earlier notebooks, are:

Input	Source / Format
lightgbm_model (1).joblib	Trained LGBMClassifier, produced by building_models.ipynb
X_test.parquet / y_test.parquet	38,172 rows × 36 features — the untouched test split from Preprocessing.ipynb

3. Step-by-Step Walkthrough of the Notebook

1 • Setup

Runs from intake import imports and installs shap and lime via `!pip install -q shap lime` directly in the notebook, then imports pandas, numpy, joblib, shap, lime.lime_tabular, and matplotlib.

2 • Load Model & Data

Loads the persisted LightGBM model and the X_test / y_test parquet files using hardcoded absolute Windows paths (e.g. C:\Users\User\Desktop\XAI_System\...). Confirms model type (LGBMClassifier) and data shape (38,172 × 36).

3 · Take a Sample

Draws a random sample of 1,000 rows from `X_test` (`random_state=42`) since SHAP's `TreeExplainer` is slow on the full test set; the notebook states 1,000 customers is enough for reliable global patterns.

4 · Compute SHAP Values

Builds a `shap.TreeExplainer` on the model and computes SHAP values for the 1,000-row sample. Correctly handles the version-dependent behavior where `LightGBM + TreeExplainer` can return a list of `[class0, class1]` arrays, selecting index 1 (the default class).

5 · Most Important Features Overall (Global)

Produces a top-10 bar chart of mean absolute SHAP value per feature — the model's overall reliance ranking — and saves it to `shap_importance.png`.

6 · Direction of Impact (Beeswarm)

Produces a full beeswarm summary plot showing, for the same top features, whether a high or low value pushes the prediction toward or away from default; saved to `shap_beeswarm.png`.

7 · Explaining ONE Customer's Prediction

Selects the customer with the single highest predicted default probability in the sample (`idx = probs.argmax()`) — 90.4% predicted risk, actual outcome = Default — and builds a SHAP waterfall plot for that one prediction, saved to `shap_waterfall.png`. The markdown frames this explicitly as "what runs in `app.py` after every prediction."

7.1 · Turning SHAP Numbers into a Sentence

Defines `explain_prediction(shap_row, feature_names, top_n=3)`, which ranks a customer's SHAP values by absolute impact and returns a plain-English bullet list (e.g. "creditscore_band decreased the risk"). The markdown explicitly flags this as the function to copy into `app.py`.

8 · LIME — A Second Opinion on the Same Customer

Builds a `LimeTabularExplainer` using the 1,000-row sample as its `training_data`, then explains the same customer selected in Step 7, saving the result to `lime_explanation.png`.

9 · Summary

Closes with a markdown table recapping each plot's purpose and a 4-step description of how the pieces connect to `app.py`: `load_model` → `predict_proba()` → `shap_values()` on that customer → `explain_prediction()` → show risk score and top 3 reasons to the user.

4. Key Findings (From Actual Executed Output)

The notebook's saved cell outputs show it ran successfully end-to-end. The measured results were extracted directly from those outputs:

Stage	Result
Model loaded	LGBMClassifier
Test set shape	38,172 rows × 36 features
SHAP/LIME sample size	1,000 rows (<code>random_state=42</code>)

SHAP computation	Succeeded for all 1,000 customers; version-mismatch warning raised and correctly handled
Example customer selected	Highest predicted risk in sample: 90.4% — actual outcome: Default
Top SHAP contributors (example customer)	creditscore_band (-1.139) · loanterm (-0.995) · employmenttype_Part-time (+0.728)
LIME run	Succeeded; raised a feature-name mismatch warning from scikit-learn/LightGBM
Artifacts saved	shap_importance.png, shap_beeswarm.png, shap_waterfall.png, lime_explanation.png (all under a local, absolute path)

5. Issues and Discrepancies Found

The notebook runs cleanly with no broken cells or undefined variables. Compared against the practices established in `Preprocessing.ipynb` and against its own stated deployment goal, a few points are worth flagging:

5.1 Hardcoded absolute local file paths

Every load and save operation (`model`, `X_test`, `y_test`, all four PNG outputs) uses an absolute Windows path such as `C:\Users\User\Desktop\XAI_System\...`, unlike `Preprocessing.ipynb`, which consistently used project-relative paths (e.g. `data/raw/`, `data/processed/`, `models/`). This will break on any other machine, in CI, or in the deployed environment, and breaks the convention set by the earlier stage.

5.2 Unclear, undocumented import

The first line of `Setup`, from `intake` import `imports`, is not explained anywhere in the notebook and doesn't appear to be used again downstream. It isn't part of the standard toolchain used elsewhere in the project (`pandas/numpy/sklearn/shap/lime`). It's worth confirming whether this is load-bearing environment setup, a leftover from a template, or a typo — and either documenting or removing it.

5.3 LIME is fit on the same small SHAP sample rather than a full reference distribution

`LimeTabularExplainer(training_data=sample.values, ...)` reuses the 1,000-row SHAP sample as LIME's own background/training data. LIME derives its feature discretization (quantile bins) from whatever is passed as `training_data`, so a 1,000-row sample may not represent the true feature distributions as well as the full `X_train` (152,952 rows after `SMOTETomek`) or even the full `X_test` (38,172 rows). This could make the notebook's LIME explanation differ from what a production call in `app.py` — built against the full reference data — would actually generate.

5.4 No random_state set for LIME

`LimeTabularExplainer`'s perturbation-based sampling is inherently randomized unless a `random_state` is supplied. None is set here, so re-running this cell may produce a visibly different set of top features or weights for the same customer, which weakens the "SHAP and LIME agree, therefore trustworthy" argument the notebook is making in Section 8.

5.5 The demo customer is a cherry-picked, high-confidence case

`idx = probs.argmax()` always selects the single highest-probability prediction in the sample. This produces a clean, easy-to-read example, but it doesn't demonstrate that SHAP/LIME explanations stay coherent for borderline predictions near 50% — arguably the cases where end users most need a trustworthy explanation, since the model itself is least confident there.

5.6 ROC-AUC and feature-importance claims are asserted, not re-derived

The intro markdown states the model's ROC-AUC = 0.74 without recomputing it in this notebook — presumably carried over from `building_models.ipynb`, but not directly traceable from this file alone. On a related note, this notebook's own SHAP output for the example customer ranks `creditscore_band` (not raw `creditscore`) as the top contributor, which is useful independent evidence supporting `Preprocessing.ipynb`'s recommendation that `creditscore_band` recovers `CreditScore`'s non-linear predictive signal — though this is based on a single customer and hasn't been verified at the global-importance level.

6. Recommendations

- **Replace hardcoded paths with project-relative paths** (e.g. via `pathlib` and a shared config), matching the convention already used in `Preprocessing.ipynb`, so the notebook is portable across machines and can run in CI/deployment.
- **Remove or document** `from intake import imports` — confirm whether it's required for the notebook to run standalone; delete it if unused, or add a comment explaining its role.
- **Fit the LIME explainer on the full reference distribution** (`X_train` or at least the full `X_test`) instead of the 1,000-row SHAP sample, and set a fixed `random_state` so LIME's output is reproducible run-to-run.
- **Add one or two borderline-probability examples** (closest to 50%) alongside the high-confidence one, to show SHAP/LIME explanations remain coherent and useful for uncertain predictions — the case end users will most want a clear reason for.
- **Report SHAP importance for `creditscore_band`, `dtiratio`, and `high_risk_flag` explicitly** at the global (top-20) level, not just for one customer, to directly close the loop with `Preprocessing.ipynb`'s open question about `CreditScore` and `DTIRatio`'s apparent absence from the top linear-correlation features.
- **Make the ROC-AUC = 0.74 figure traceable** — either link to the exact evaluation cell in `building_models.ipynb` or restate/recompute it here after loading the model, so the number can't silently drift out of sync if the model is retrained.

Overall assessment: `shap_lime_explainer.ipynb` is a small, focused, and functionally correct notebook that successfully bridges model output and end-user-facing explanation text, and it runs without structural bugs. The issues found are mainly portability (hardcoded paths), reproducibility (unseeded LIME, small LIME reference sample), and documentation-traceability concerns rather than logic errors, and are worth resolving before the `explain_prediction()` function and file paths are copied directly into `app.py` as the notebook itself proposes.

Report generated by reviewing `shap_lime_explainer.ipynb` (22 cells) — code and previously saved cell outputs — for the `depi_loan_default_xai` project.